Informe comparativo de 12 arquitecturas de software

Trabajo en Clase 02 — Ingeniería de Software III

Diego López Camacho — Universidad Industrial de Santander

23 de septiembre de 2026

1. Introducción

Elegir una arquitectura no es elegir la que está de moda, sino la que mejor resuelve las tensiones concretas de un sistema. Toda decisión arquitectónica mejora unos atributos de calidad a costa de otros: hacer un sistema más escalable casi siempre lo hace más difícil de operar, y hacerlo más simple de desplegar suele limitar hasta dónde puede crecer. Por eso el criterio de comparación no puede ser "cuál es mejor", sino "qué atributo privilegia cada una y a cambio de qué".

Para mantener la comparación homogénea, este informe evalúa las doce arquitecturas frente a seis atributos de calidad tomados del modelo de la norma ISO/IEC 25010 y de la clasificación de Bass, Clements y Kazman (2021):

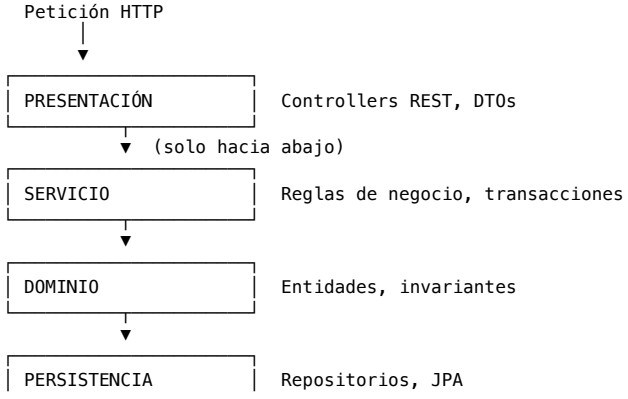
Mantenibilidad: facilidad para entender el código, localizar dónde cambiar algo y modificarlo sin romper lo demás; depende de la claridad de los límites internos y de la dirección de las dependencias. **Escalabilidad:** capacidad de absorber más carga agregando recursos, distinguiendo si se escala el sistema completo o solo la parte saturada. **Desacoplamiento:** grado en que un módulo cambia sin obligar a cambios en otros; depender de una implementación concreta acopla, depender de un contrato o de un evento no.

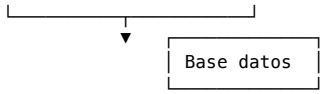
Testabilidad: facilidad para probar la lógica de negocio sin levantar infraestructura, es decir, ejercitando el dominio con dobles de prueba en lugar de base de datos, red o colas reales. **Capacidad de despliegue:** esfuerzo y riesgo de poner un cambio en producción, medido por la duración del ciclo compilar-empaquetar-desplegar y por el alcance del impacto. **Complejidad operativa:** costo de mantener el sistema funcionando —observabilidad, trazas distribuidas, configuración, manejo de fallos parciales—; es el atributo que más se subestima, el que paga la factura de las arquitecturas distribuidas y el único en el que un valor alto es desfavorable.

Estos seis atributos son el lenguaje común del informe: se califican de forma homogénea en la tabla de la sección 14 y se usan en el caso aplicado de la sección 15 para justificar por qué, en un sistema académico universitario, no todas las opciones son razonables.

2. Arquitectura en Capas

Definición y principios. Organiza el sistema en niveles horizontales donde cada nivel agrupa responsabilidades técnicas del mismo tipo y solo se comunica con el inmediatamente inferior. El principio que la sostiene es la separación por responsabilidad técnica: la presentación no sabe cómo se persisten los datos y la persistencia no sabe quién la invoca. Richards y Ford (2020) la describen como la arquitectura por defecto de la industria, en el sentido de que aparece cuando un equipo no toma una decisión arquitectónica explícita. Una distinción importante es la de capas cerradas frente a abiertas: en el esquema cerrado una petición debe atravesar todas las capas en orden, sin saltos, y eso es justamente lo que protege el aislamiento, porque si la presentación pudiera llamar a la persistencia, la capa de servicio dejaría de ser el único lugar donde viven las reglas.





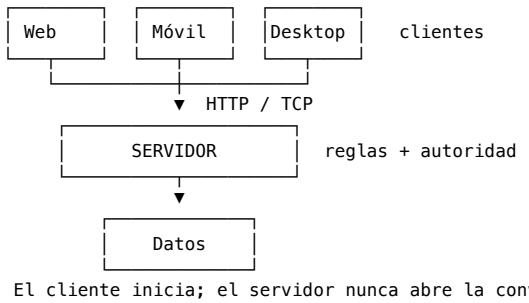
Ventajas y desventajas. Su mayor ventaja es organizativa: cualquier desarrollador que llega sabe dónde buscar, el costo de entrada es bajo y la depuración es lineal. Al concentrar las reglas en una capa, evita que la lógica se disperse entre controladores y consultas SQL. Las desventajas aparecen con el tamaño. La más citada es el sumidero de cambios: agregar un campo obliga a tocar las cuatro capas, convirtiendo un cambio trivial en una modificación transversal. A esto se suma que el dominio termina dependiendo de la persistencia —las entidades llevan anotaciones de JPA—, de modo que la regla de negocio queda atada a la tecnología de almacenamiento, y que el despliegue es una sola unidad.

Casos de uso. Aplicaciones empresariales de alcance acotado, sistemas internos con reglas moderadas, equipos pequeños y, de forma destacada, cualquier sistema que esté arrancando y cuyos límites de dominio aún no se conocen. Es también la arquitectura adecuada en contextos de aprendizaje, porque hace visibles las responsabilidades antes de introducir indirecciones.

Cuándo NO usarla. Cuando se deben escalar partes de forma independiente, cuando la lógica de negocio es el activo principal y debe sobrevivir a cambios de tecnología, o cuando varios equipos necesitan desplegar sin coordinarse. Tampoco si el dominio es muy rico: la capa de servicio crece hasta volverse un conjunto de procedimientos largos y el dominio queda reducido a estructuras de datos sin comportamiento, resultado conocido como modelo de dominio anémico (Fowler, 2002).

3. Arquitectura Cliente–Servidor

Definición y principios. Es un estilo de distribución —no de organización interna— que reparte responsabilidades entre un proceso que solicita servicios y uno que los provee. El principio central es la asimetría de roles con un contrato de comunicación explícito: el cliente inicia la interacción, el servidor la atiende, y ninguno conoce la implementación interna del otro, solo el protocolo. De aquí se deriva una consecuencia que suele pasarse por alto: el servidor es un punto de autoridad, y centralizar datos y reglas en él es lo que permite que clientes heterogéneos —web, móvil, escritorio— se comporten de forma consistente.



Ventajas y desventajas. Centralizar permite controlar: seguridad, integridad y reglas se aplican en un solo lugar, y actualizar la lógica no obliga a actualizar cada cliente instalado. En contra, el servidor es un punto único de falla y un cuello de botella natural: si se cae, todos los clientes quedan inutilizables. La latencia de red pasa a formar parte del comportamiento percibido, y el modelo clásico de dos niveles escala mal porque todas las conexiones compiten por los mismos recursos.

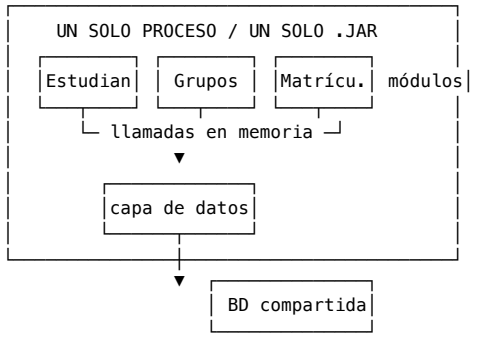
Casos de uso. Es la base de prácticamente toda aplicación web y de toda API REST, incluido el proyecto `demo-academico` de este curso. También aplica a sistemas de bases de datos, correo electrónico y cualquier escenario que requiera una fuente de verdad compartida.

Cuándo NO usarla. Cuando se requiere operación fuera de línea con plena funcionalidad, cuando la comunicación debe ser entre pares sin coordinador (redes P2P), o cuando la latencia hacia un servidor central es inaceptable. En esos casos se recurre a arquitecturas descentralizadas o a modelos *offline-first* con reconciliación posterior.

4. Arquitectura Monolítica

Definición y principios. Un monolito se compila, empaqueta y despliega como una sola unidad ejecutable. El principio no es la ausencia de estructura interna —un monolito puede estar perfectamente modularizado— sino la unidad de despliegue: todas las partes comparten proceso, memoria y ciclo de vida. Conviene distinguir dos variantes que se confunden sistemáticamente. El monolito desordenado no tiene límites internos: cualquier clase llama a cualquier otra y el acoplamiento crece sin control. El monolito modular define módulos con interfaces públicas y prohíbe los accesos que las esquivan; mantiene la simplicidad de despliegue y gana la disciplina de dependencias.

Richards y Ford (2020) proponen esta segunda variante como la alternativa realista a los microservicios para la mayoría de sistemas.



Modular → se hablan por interfaces. Desordenado → por donde sea.

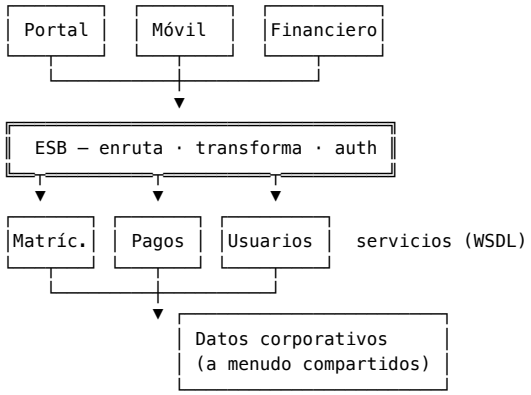
Ventajas y desventajas. Es la arquitectura con menor complejidad operativa que existe: un artefacto, un despliegue, un log, una transacción. Las llamadas entre módulos son invocaciones en memoria, sin serialización ni fallos de red, y la consistencia transaccional se obtiene gratis; depurar es seguir una pila de llamadas, no correlacionar trazas. El precio se paga en granularidad: no se puede escalar solo la parte saturada, de modo que si matrículas se congestiona en temporada de inscripciones hay que replicar toda la aplicación. El acoplamiento tiende a crecer si no se vigila, los tiempos de compilación y arranque aumentan con el tamaño, todo queda atado a un único *stack*, y una fuga de memoria en un módulo secundario puede tumbar el proceso completo.

Casos de uso. Sistemas nuevos cuyos límites de dominio aún no están claros, equipos pequeños o medianos y aplicaciones de carga predecible. Newman (2021) sostiene explícitamente que el monolito debería ser el punto de partida por defecto y que migrar solo se justifica ante un problema concreto que el monolito no pueda resolver.

Cuándo NO usarla. Cuando distintas partes tienen perfiles de carga radicalmente distintos, cuando varios equipos necesitan desplegar de forma independiente y la coordinación se vuelve el cuello de botella, o cuando hay una exigencia real de usar tecnologías diferentes por subdominio.

5. Arquitectura Orientada a Servicios (SOA)

Definición y principios. SOA organiza las capacidades del negocio como servicios reutilizables con contratos formales, publicados en un registro y accesibles por toda la organización. Erl (2005) lo plantea como una estrategia de integración de alcance empresarial más que como un patrón de diseño de una aplicación: el objetivo es que la capacidad “validar matrícula” exista una sola vez y la consuman todos los sistemas que la necesiten. Sus principios distintivos son el contrato formal y versionado (históricamente WSDL/SOAP), la reutilización empresarial, la composición de servicios para formar procesos de negocio, y la mediación mediante un bus de integración que centraliza enrutamiento, transformación de mensajes y seguridad.



Ventajas y desventajas. Su fortaleza es integrar un panorama heterogéneo: SOA fue la respuesta al problema real de organizaciones con decenas de sistemas legados que debían hablar entre sí. El contrato formal y el versionado permiten evolucionar un servicio sin romper a sus consumidores, y la gobernanza centralizada da trazabilidad y control de seguridad. Sus debilidades son el peso y la centralización: el ESB, pensado para desacoplar, acaba concentrando lógica de negocio y volviéndose cuello de botella técnico y organizativo, porque cada cambio pasa por el equipo de integración. Los servicios suelen ser de grano grueso, los protocolos SOAP/XML son verbosos, y compartir la base de datos entre servicios reintroduce el acoplamiento que se quería eliminar.

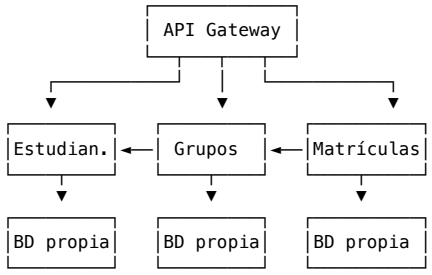
Casos de uso. Grandes organizaciones con sistemas heredados que deben integrarse, sectores con requisitos

normativos fuertes de trazabilidad y contratos —banca, seguros, sector público— y escenarios donde reutilizar capacidades entre áreas es un objetivo explícito del negocio.

Cuándo NO usarla. En una aplicación nueva de alcance limitado el costo de gobernanza, registro y bus supera por mucho el beneficio. Tampoco en equipos pequeños, ni cuando se busca autonomía de despliegue: la mediación centralizada es justamente lo contrario.

6. Arquitectura de Microservicios

Definición y principios. La aplicación se construye como un conjunto de servicios pequeños, desplegables de forma independiente, cada uno dueño de sus datos y organizado alrededor de una capacidad de negocio. Newman (2021) insiste en que la propiedad definitoria no es el tamaño sino la independencia de despliegue: si no se puede desplegar un servicio sin coordinar con otros, no se obtienen los beneficios del estilo, solo sus costos. Cuatro principios lo hacen funcionar: cada servicio es dueño exclusivo de su almacenamiento, sin base de datos compartida; los límites se trazan sobre contextos de negocio y no sobre capas técnicas; la comunicación es por red mediante contratos explícitos; y el diseño asume que los fallos parciales son normales, no excepcionales.



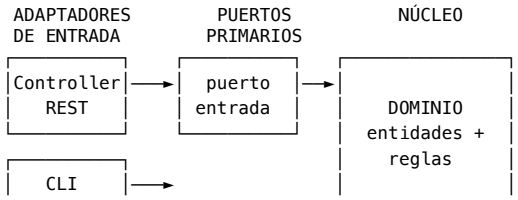
Ventajas y desventajas. Permite lo que ninguna arquitectura monolítica permite: escalar solo lo saturado, desplegar sin tocar el resto, aislar fallos y elegir tecnología por servicio; organizativamente habilita equipos autónomos avanzando en paralelo. El costo es la complejidad distribuida, y es alto. La consistencia transaccional desaparece: una operación que abarca dos servicios ya no puede ser una transacción y hay que resolverla con sagas o consistencia eventual. Depurar exige trazas distribuidas. Aparecen problemas que antes no existían: latencia acumulada, reintentos, idempotencia, versionado de contratos, descubrimiento de servicios. Y si los límites se trazan mal el resultado es un monolito distribuido: todos los inconvenientes de la red sin ninguna ventaja de la independencia.

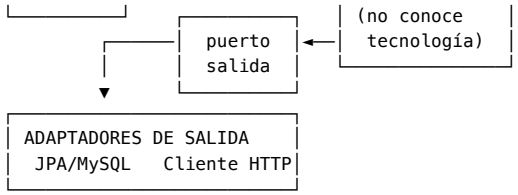
Casos de uso. Sistemas de gran escala con partes de carga muy dispar, organizaciones con múltiples equipos que necesitan desplegar sin coordinación, y dominios donde ciertos subsistemas tienen requisitos de disponibilidad o tecnología claramente distintos. Netflix y Uber han documentado públicamente su adopción por razones de escala y autonomía organizativa.

Cuándo NO usarla. En un sistema nuevo cuyos límites de dominio no se conocen —equivocarse al partir es mucho más costoso que no haber partido—, en equipos pequeños que no pueden sostener la carga operativa, cuando no hay madurez en automatización y observabilidad, o cuando el dominio exige consistencia fuerte en operaciones que cruzan varios servicios.

7. Arquitectura Hexagonal (Ports & Adapters)

Definición y principios. Propuesta por Cockburn (2005), coloca el dominio en el centro y lo aísla de toda tecnología mediante puertos —interfaces que el dominio define— y adaptadores —implementaciones que los conectan con el exterior—. La regla fundamental es la dirección de las dependencias: siempre apuntan hacia el núcleo, nunca hacia afuera. La distinción clave es entre puertos primarios y secundarios. Los primarios son la API que el dominio ofrece; los secundarios son lo que el dominio necesita del exterior, declarado en sus propios términos: `RepositorioDeEstudiantes`, no `JpaRepository`. Con esto la base de datos deja de ser el centro del sistema y pasa a ser un detalle reemplazable, igual que la interfaz de usuario. Es exactamente la idea que aplicamos en el Lab 03 del curso: el módulo de matrículas no inyecta `EstudianteRepository` ni `GrupoRepository`, sino los puertos `EstudianteQueryPort` y `GrupoQueryPort`.





Las flechas SIEMPRE apuntan al dominio.

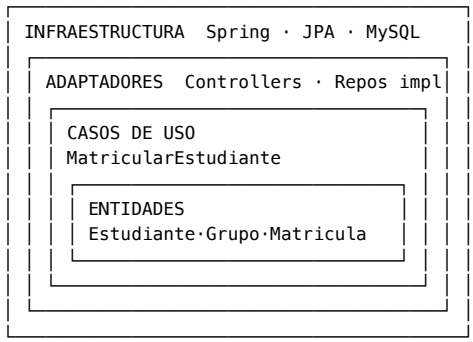
Ventajas y desventajas. Su mayor virtud es la testabilidad: el dominio se prueba sin base de datos, sin servidor web y sin red, reemplazando los adaptadores por dobles en memoria, lo que convierte pruebas de minutos en pruebas de milisegundos. Además la tecnología se vuelve sustituible —cambiar JPA afecta a un adaptador, no al dominio— y las reglas quedan explícitas en un solo lugar, protegidas de la erosión que produce mezclarlas con infraestructura. Sus desventajas son el número de archivos y la curva de aprendizaje: para un CRUD simple, un puerto y un adaptador por cada acceso a datos es una indirección que no paga. También exige disciplina sostenida, porque basta que un desarrollador importe el repositorio directamente para que el aislamiento se rompa sin que nada falle visiblemente.

Casos de uso. Sistemas con lógica de negocio rica y de larga vida, donde el dominio es el activo que debe sobrevivir a los cambios de tecnología; también cuando se anticipa cambiar de proveedor de infraestructura o cuando la cobertura de pruebas rápidas del dominio es un requisito.

Cuándo NO usarla. En un CRUD sin reglas, en prototipos o pruebas de concepto, y en equipos sin experiencia en inversión de dependencias: aplicada a medias produce la peor combinación, más archivos sin ganar aislamiento real.

8. Clean Architecture

Definición y principios. Formulada por Martin (2017), organiza el sistema en anillos concéntricos con una única regla: el código de un anillo interior no puede conocer nada de un anillo exterior. De adentro hacia afuera son entidades —reglas de negocio independientes de cualquier aplicación—, casos de uso —reglas propias de esta aplicación—, adaptadores de interfaz e infraestructura. Comparte con la hexagonal el objetivo de aislar el dominio, pero añade dos cosas: estratifica el interior, separando lo que es regla del negocio en sí (una matrícula no puede exceder el cupo) de lo que es regla de este caso de uso concreto (al matricular, validar primero existencia, luego duplicado, luego cupo); y hace explícita la inversión de dependencias en el cruce de fronteras, porque cuando un caso de uso necesita la base de datos declara él mismo la interfaz y la infraestructura la implementa.



Dependencias: ⟶ hacia adentro. Nunca ⟵ hacia afuera.

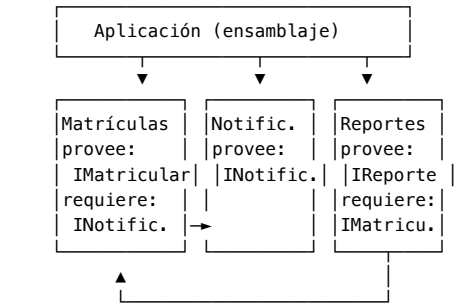
Ventajas y desventajas. Produce sistemas donde el negocio es verdaderamente independiente del *framework*: se puede cambiar Spring por otro contenedor, o la web por una CLI, sin tocar casos de uso ni entidades; los casos de uso documentan el comportamiento de forma legible y la testabilidad es máxima. En contra, es la arquitectura con mayor sobrecarga estructural de las estudiadas: multiplica los objetos de transferencia, de modo que un mismo dato puede existir como entidad, como modelo de petición del caso de uso, como modelo de respuesta y como DTO de la API, con todas las conversiones correspondientes. Para dominios simples es desproporcionado, y sin convenciones claras la cantidad de capas genera discusiones improductivas sobre dónde va cada cosa.

Casos de uso. Sistemas de dominio complejo y vida larga donde el costo de la estructura se amortiza —plataformas financieras, sistemas de salud, *core* de seguros— y proyectos con alta rotación de personal, porque la estructura rígida reduce la ambigüedad.

Cuándo NO usarla. En aplicaciones pequeñas, en servicios cuya razón de ser es exponer datos con poca transformación, o cuando el equipo prioriza velocidad de entrega inicial. Aplicarla por prestigio y no por necesidad es una de las formas más comunes de sobreingeniería.

9. Arquitectura Basada en Componentes

Definición y principios. Construye el sistema ensamblando unidades de software reemplazables con interfaces bien definidas. Szyperski (2002) define el componente por tres propiedades: es una unidad de despliegue independiente, no tiene estado observable desde fuera, y se compone con otros únicamente a través de contratos explícitos. La idea de fondo es que el software se arme como se arma hardware, con piezas intercambiables de las que solo importa la especificación. El principio operativo es que un componente oculta por completo su implementación y declara tanto lo que ofrece —interfaces provistas— como lo que requiere —interfaces requeridas—; esa segunda parte es la que permite el ensamblaje, porque un contenedor conecta las piezas resolviendo las dependencias declaradas.



Solo se conocen por interfaces (I...).
Cambiar un componente = respetar su interfaz.

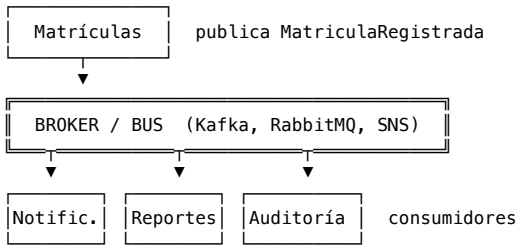
Ventajas y desventajas. Favorece la reutilización real y permite sustituir una pieza sin afectar al resto siempre que el contrato se mantenga; al ser unidades aisladas se desarrollan y prueban en paralelo, y el versionado por componente da un control de la evolución más fino. Sus dificultades son el diseño de las interfaces y la gestión de versiones: definir un contrato estable y a la vez útil es difícil, porque si queda demasiado específico no se reutiliza y si queda demasiado genérico no sirve. Cuando el número de componentes crece, la red de dependencias y versiones compatibles se vuelve un problema en sí mismo, y la reutilización prometida se concreta menos de lo esperado, porque dos contextos que parecen necesitar lo mismo casi siempre difieren en algún detalle.

Casos de uso. Plataformas extensibles mediante complementos —IDEs, navegadores, gestores de contenido—, bibliotecas y *frameworks*, líneas de producto donde varias versiones comparten piezas, y sistemas embebidos con módulos certificados por separado.

Cuándo NO usarla. Cuando no hay intención real de reutilizar o sustituir piezas, porque el esfuerzo de diseñar contratos estables se desperdicia. Tampoco en dominios muy volátiles, donde las interfaces cambiarían tan seguido que dejarían de cumplir su función de estabilidad.

10. Arquitectura Orientada a Eventos (EDA)

Definición y principios. Los componentes se comunican publicando hechos ya ocurridos, sin saber quién los va a consumir. El principio central es la inversión del conocimiento: el productor no conoce a sus consumidores. `MatriculaRegistrada` se publica una vez y quien necesite reaccionar —notificaciones, reportes, auditoría— se suscribe sin que el productor se entere. Hohpe y Woolf (2003) distinguen dos topologías: en la de mediador un orquestador recibe un evento inicial y coordina los pasos de un proceso de negocio, útil cuando hay orden y necesidad de compensación; en la de difusión el productor publica en un canal y los consumidores actúan de forma autónoma, lo que da el máximo desacoplamiento pero deja el flujo global sin estar escrito en ningún lugar. También conviene separar la notificación de evento —un aviso mínimo, y el consumidor consulta los detalles— de la transferencia de estado, donde el evento lleva toda la información: la primera mantiene los eventos pequeños a costa de llamadas de vuelta, la segunda las evita duplicando datos.



El productor NO sabe quiénes escuchan.
Agregar un consumidor = cero cambios arriba.

Ventajas y desventajas. Es la arquitectura que mejor soporta la extensión sin modificación: agregar un consumi-

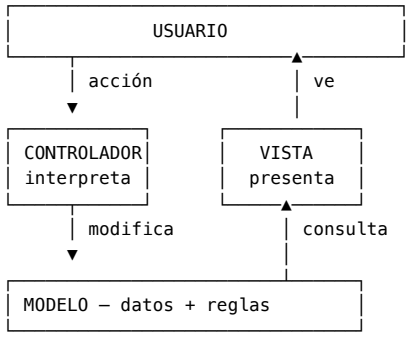
dor no requiere tocar al productor, lo que en sistemas que crecen por acumulación de reacciones a los mismos hechos es una ventaja enorme. El procesamiento asíncrono absorbe picos porque la cola amortigua, y los consumidores se escalan de forma independiente. Los inconvenientes son serios: el flujo completo del negocio no está escrito en ningún archivo y hay que reconstruirlo leyendo quién publica y quién escucha, lo que dificulta el razonamiento y la incorporación de nuevos desarrolladores. La consistencia es eventual por diseño, el sistema puede quedar temporalmente en estados intermedios visibles, depurar exige correlación de eventos, y hay que diseñar explícitamente para entrega duplicada (idempotencia), orden no garantizado y eventos que fallan repetidamente.

Casos de uso. Sistemas con muchas reacciones a un mismo hecho, procesamiento de flujos en tiempo real (IoT, telemetría, detección de fraude), integración desacoplada entre sistemas de distinta velocidad, y escenarios donde se requiere el historial completo de cambios como fuente de verdad (*event sourcing*).

Cuándo NO usarla. Cuando se exige consistencia inmediata y confirmación sincrónica al usuario, cuando el flujo es simple y lineal —convertirlo en eventos solo lo oscurece—, o cuando el equipo no puede operar un *broker* con sus garantías de entrega y monitoreo.

11. Modelo–Vista–Controlador (MVC)

Definición y principios. MVC es un patrón de organización de la interacción con el usuario, no una arquitectura de sistema completo. Descrito por Krasner y Pope (1988) para Smalltalk-80, separa tres responsabilidades: el modelo mantiene datos y reglas, la vista presenta la información y el controlador traduce las acciones del usuario en operaciones sobre el modelo. Es importante señalar que el MVC original y el de los *frameworks* web actuales no son lo mismo: en el original la vista observa al modelo y se actualiza cuando este cambia, con flujo circular; en el MVC web el ciclo petición-respuesta de HTTP impone un flujo lineal, donde el controlador atiende, actualiza el modelo y selecciona una vista que se renderiza una sola vez, sin observación continua porque no hay estado persistente entre peticiones.



MVC clásico: la vista OBSERVA al modelo.
MVC web: flujo lineal petición → respuesta.

Ventajas y desventajas. Separa con claridad la presentación de la lógica, lo que permite varias vistas del mismo modelo —HTML, JSON, PDF— y cambiar la interfaz sin tocar el negocio; es un vocabulario universalmente conocido y está soportado por casi todos los *frameworks* web. Sus problemas aparecen cuando se lo usa como si fuera la arquitectura completa. El síntoma típico es el controlador gordo: como MVC no dice dónde va la lógica de negocio, esta termina en el controlador, que crece hasta volverse inmantenible. En interfaces muy interactivas la relación entre vistas y controladores se vuelve difícil de seguir, y en el MVC clásico la notificación modelo-vista genera actualizaciones en cascada difíciles de depurar; de ahí surgieron variantes como MVP y MVVM.

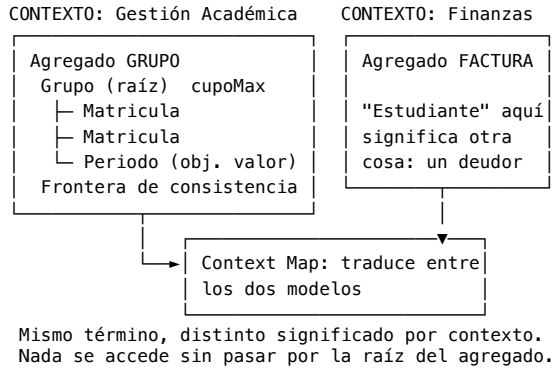
Casos de uso. Aplicaciones web con renderizado en servidor, APIs REST en la capa de entrada, interfaces de escritorio y en general cualquier punto donde haya que mediar entre acciones del usuario y un modelo de datos.

Cuándo NO usarla. No aplica a sistemas sin interfaz de usuario, como procesos por lotes o servicios de integración. Y no debe usarse como sustituto de una decisión arquitectónica: organiza la entrada, pero no responde dónde viven las reglas de negocio ni cómo se aíslan de la infraestructura. En `demo-academico`, `@RestController` cumple el rol de controlador, pero la arquitectura del sistema es en capas y, tras el Lab 03, modular con puertos.

12. Domain-Driven Design (DDD)

Definición y principios. DDD, formulado por Evans (2003), es antes que nada un enfoque de modelado que pone el dominio del negocio en el centro del diseño, y del que se derivan consecuencias arquitectónicas. Su aporte más duradero es el diseño estratégico: la idea de que un sistema grande no tiene un único modelo coherente sino varios contextos acotados, cada uno con su propio modelo y vocabulario. El lenguaje ubicuo mantiene alineados modelo y negocio, porque los términos del código son los mismos que usan los expertos del dominio. En el diseño táctico distingue entidades —identidad propia que persiste—, objetos de valor —definidos solo por sus atributos—,

agregados —grupos con una raíz que es el único punto de entrada y la frontera de consistencia transaccional—, repositorios y servicios de dominio. El agregado es el concepto de mayor impacto arquitectónico: define qué debe ser consistente de inmediato y qué puede serlo eventualmente. Si `Grupo` es la raíz del agregado que contiene las matrículas, la regla de cupo se verifica en una sola transacción; si `Matricula` es su propio agregado, hay que resolverla de otra manera.



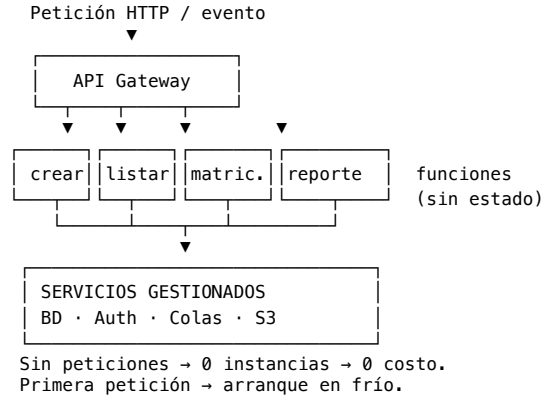
Ventajas y desventajas. Produce modelos que el negocio reconoce, lo que reduce drásticamente los malentendidos de requisitos; los contextos acotados dan un criterio sólido para trazar límites de módulos o servicios, muy superior al criterio técnico de separar por capas, y los agregados hacen explícitas las reglas de consistencia en lugar de dejarlas implícitas. Sus costos son altos y de naturaleza organizativa más que técnica: exige acceso sostenido a expertos del dominio, y sin eso DDD degenera en nomenclatura vacía. La curva de aprendizaje es pronunciada y el vocabulario se adopta con frecuencia de forma superficial. Para dominios simples es desproporcionado, porque aplicar agregados y objetos de valor a un CRUD de catálogos añade estructura sin ningún beneficio.

Casos de uso. Dominios complejos donde la lógica de negocio es la verdadera dificultad del proyecto —seguros, logística, salud, banca— y especialmente como paso previo a una partición en microservicios, porque los contextos acotados son el criterio correcto para decidir dónde cortar.

Cuándo NO usarla. Cuando el problema es fundamentalmente técnico y no de dominio, como un servicio de transcodificación o un *gateway*; cuando no hay expertos de negocio disponibles; o en aplicaciones cuya esencia es almacenar y mostrar datos sin reglas significativas.

13. Arquitectura Serverless

Definición y principios. El proveedor gestiona por completo la infraestructura de ejecución y el código se organiza en funciones de corta vida activadas por eventos: el servidor no desaparece, desaparece la responsabilidad de administrarlo. Roberts y Chapin (2017) distinguen FaaS —funciones como servicio, el cómputo propio— de BaaS —servicios gestionados de terceros para base de datos, autenticación o almacenamiento—. Dos principios la caracterizan. Las funciones no guardan estado: cualquier dato que deba sobrevivir a la invocación va a un servicio externo. Y el escalado llega hasta cero: si no hay peticiones no hay instancias y por tanto no hay costo. Esto cambia el modelo económico, porque se paga por invocación y tiempo de cómputo en lugar de capacidad reservada, pero introduce el arranque en frío: la primera invocación tras un periodo de inactividad paga el costo de inicializar el entorno, que para aplicaciones JVM tradicionales puede ser de segundos.



Ventajas y desventajas. Elimina la administración de servidores, parches y dimensionamiento de capacidad, y el escalado es automático desde cero hasta miles de ejecuciones concurrentes; para cargas intermitentes el ahorro

es real, porque un proceso que corre una vez al día no paga por las otras veintitrés horas. En contra, el arranque en frío introduce latencia impredecible, lo que descalifica el modelo para rutas sensibles al tiempo de respuesta si no se mitiga. Hay límites duros de duración, memoria y tamaño del paquete; la dependencia del proveedor es alta porque las funciones se integran con sus servicios gestionados; probar localmente es difícil y depurar un flujo que atraviesa varias funciones requiere trazas distribuidas. Finalmente, para carga constante y alta resulta más caro que una instancia reservada.

Casos de uso. Tareas programadas y procesos por lotes, procesamiento de archivos disparado por carga, *webhooks* e integraciones, APIs de tráfico bajo o muy variable y procesamiento de flujos de eventos. Es un buen complemento de una arquitectura principal más que un sustituto de ella.

Cuándo NO usarla. Para cargas constantes y predecibles de alto volumen, operaciones de larga duración que exceden los límites del proveedor, cuando la latencia debe ser estable y baja, cuando se requiere portabilidad entre nubes, o en aplicaciones con estado en memoria y conexiones persistentes.

14. Tabla comparativa general

Las calificaciones son relativas entre sí, no absolutas: "Baja" en escalabilidad no significa que el sistema no escale, sino que el estilo no ofrece mecanismos propios para escalar partes de forma independiente. En **complejidad operativa**, a diferencia del resto de columnas, un valor alto es desfavorable.

Arquitectura	Mantenib.	Escalab.	Desacopl.	Testab.	Despliegue	Compl. oper.
En Capas	Media	Baja	Bajo	Media	Baja	Baja
Cliente–Servidor	Media	Media	Medio	Media	Media	Baja
Monolítica	Baja	Baja	Bajo	Media	Baja	Muy baja
SOA	Media	Media	Medio	Baja	Baja	Alta
Microservicios	Alta	Muy alta	Alto	Media	Muy alta	Muy alta
Hexagonal	Alta	Media	Alto	Muy alta	Media	Baja
Clean Architecture	Muy alta	Media	Muy alto	Muy alta	Media	Media
Basada en Componentes	Alta	Media	Alto	Alta	Media	Media
Orientada a Eventos	Alta	Alta	Muy alto	Baja	Alta	Alta
MVC	Media	Baja	Medio	Media	Baja	Baja
DDD	Muy alta	Media	Alto	Alta	Media	Media
Serverless	Media	Muy alta	Alto	Baja	Muy alta	Media

Conviene añadir un criterio que la tabla no recoge y que explica buena parte de sus calificaciones: **con qué se trazan los límites internos**. En Capas y en MVC se parte por responsabilidad técnica; en Monolítica modular, Hexagonal y Clean Architecture por módulo de negocio o por anillo de dependencia; en SOA por capacidad empresarial; en Microservicios y DDD por contexto de negocio; en EDA por evento; y en Serverless por operación individual. Partir por criterio técnico produce módulos que siempre cambian juntos; partir por contexto de negocio produce módulos que cambian por separado.

Hay dos observaciones que la tabla deja ver y que conviene hacer explícitas. La primera es que **no todas las filas son comparables entre sí**: MVC es un patrón de la capa de entrada, DDD es un enfoque de modelado y Cliente–Servidor es un estilo de distribución. Ninguno de los tres compite con Microservicios o con Hexagonal, y de hecho se combinan con ellos: un sistema puede ser simultáneamente cliente-servidor, hexagonal, modelado con DDD y usar MVC en su borde REST. Tratarlas como opciones mutuamente excluyentes es un error de categoría.

La segunda es que **la complejidad operativa es el costo oculto** de las arquitecturas que mejor puntúan en escalabilidad y despliegue. Microservicios obtiene las mejores calificaciones en esas dos columnas y la peor en la última, y la correlación no es accidental: la independencia de despliegue se compra con infraestructura de observabilidad, automatización y manejo de fallos parciales que alguien tiene que construir y mantener.

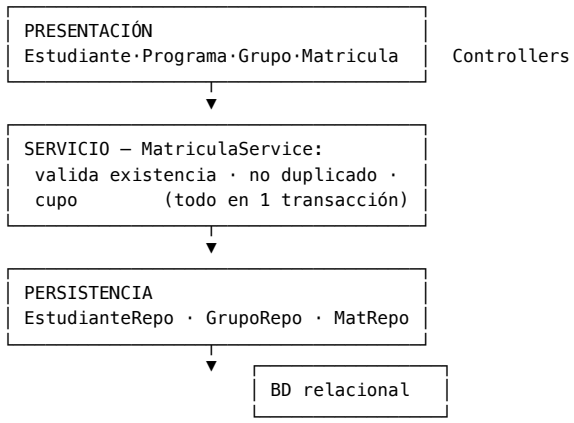
15. Caso aplicado: Sistema Académico

Se toma como caso un **sistema académico universitario** que gestiona programas, asignaturas, grupos, estudiantes y matrículas —el mismo dominio del proyecto `demo-academico` del curso—. La regla de negocio más

representativa es la de matrícula: un estudiante no puede matricularse dos veces en el mismo grupo y un grupo no puede exceder su cupo máximo. Esta sección analiza cómo se organizaría ese sistema en tres de las arquitecturas estudiadas y referencia ejemplos reales de cada estilo. No se implementa ninguna: el propósito es comparar decisiones de diseño y sus consecuencias.

15.1 Opción A — Arquitectura en Capas

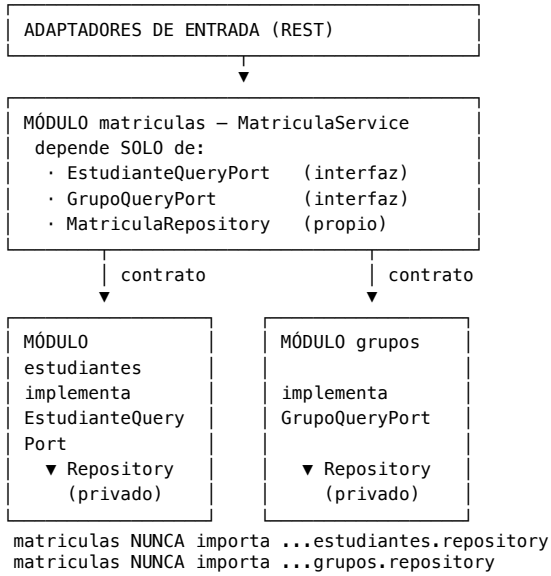
Ejemplo real de referencia. Moodle, la plataforma de gestión de aprendizaje de mayor adopción en el ámbito universitario, está construida como un monolito organizado en capas, con separación entre presentación, lógica y acceso a datos documentada en su guía de desarrollo. Es un caso útil porque demuestra que una arquitectura en capas puede sostener un sistema académico de escala institucional durante dos décadas.



El servicio de matrícula inyecta los tres repositorios y ejecuta las validaciones en una sola transacción. Es la solución más directa y la que menos código requiere. Su problema arquitectónico es visible en el diagrama: `MatriculaService` depende de los repositorios de estudiantes y de grupos, es decir, **conoce cómo se persisten entidades que no le pertenecen**. Si mañana el acceso a estudiantes cambia —pasa a consultarse a un servicio externo de la universidad, por ejemplo—, hay que modificar el servicio de matrículas, que no tenía por qué enterarse. Este es exactamente el acoplamiento que corrige el Lab 03 del curso.

15.2 Opción B — Hexagonal con módulos y puertos

Ejemplo real de referencia. El propio proyecto `demo-academico` en su versión del Lab 03 aplica puertos y adaptadores: el módulo `matriculas` tiene prohibido inyectar `EstudianteRepository` o `GrupoRepository` y debe comunicarse mediante los puertos `EstudianteQueryPort` y `GrupoQueryPort`, implementados por los servicios de los módulos dueños.

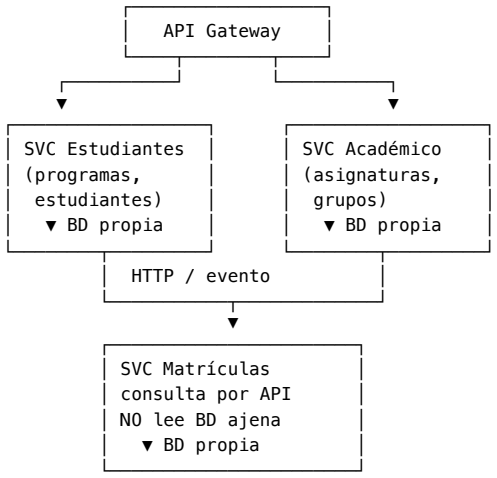


El cambio respecto a la opción A es pequeño en código y grande en consecuencias. El módulo de matrículas sigue ejecutándose en el mismo proceso y dentro de la misma transacción, pero ya no depende de la implementación de persistencia ajena sino de un contrato. Esto habilita dos cosas: la regla de cupo se puede probar sustituyendo

los puertos por dobles en memoria, sin base de datos; y si en el futuro se decide extraer el módulo de estudiantes a un servicio aparte, el cambio se concentra en escribir un adaptador que implemente `EstudianteQueryPort` llamando por HTTP, sin tocar la lógica de matrícula. Mantiene el costo operativo de un monolito —un despliegue, una transacción, un log— mientras obtiene el desacoplamiento de una arquitectura modular.

15.3 Opción C — Microservicios

Ejemplo real de referencia. Netflix y Uber han documentado públicamente en sus blogs de ingeniería la migración de un monolito a cientos de servicios independientes, motivada por la necesidad de escalar partes del sistema de forma dispar y de permitir que decenas de equipos desplieguen sin coordinarse. Son los casos canónicos del estilo y sirven también para ilustrar cuándo *sí* se justifica.



La validación de cupo deja de ser transaccional: requiere reserva previa de cupo o una saga.

Aquí aparece el problema de fondo. La regla “no superar el cupo” deja de resolverse con una transacción, porque el conteo de matrículas vive en el servicio de matrículas y el cupo máximo vive en el servicio académico. Dos peticiones concurrentes pueden leer el mismo conteo y ambas concluir que hay cupo. Resolverlo exige mecanismos adicionales: una reserva de cupo en el servicio dueño del grupo, o una saga con compensación si la matrícula falla después de reservar.

Es decir: el microservicio no simplifica la regla de negocio, la complica. Y a cambio ofrece algo que un sistema académico universitario rara vez necesita —escalar matrículas con independencia de estudiantes— y que además tiene carácter estacional, porque el pico de carga ocurre unas semanas al año, lo que se resuelve con mucho menos esfuerzo replicando un monolito durante ese periodo.

15.4 Decisión y justificación

Criterio	Capas	Hexagonal + módulos	Microservicios
Esfuerzo inicial	Muy bajo	Bajo	Muy alto
Testabilidad de la regla de cupo	Requiere BD	Sin BD, con dobles	Requiere entorno completo
Consistencia del cupo	Transaccional	Transaccional	Eventual / saga
Acoplamiento entre módulos	Alto	Bajo	Bajo
Complejidad operativa	Muy baja	Baja	Muy alta
Equipo necesario	1–3 personas	1–5 personas	Varios equipos

Para este caso la opción adecuada es la **B: hexagonal con módulos y puertos**. La razón no es que sea la más sofisticada, sino que es la única que mejora el atributo que realmente está en juego —el desacoplamiento entre módulos y la testabilidad de las reglas— sin pagar el costo que no se necesita.

La opción A funciona y es un punto de partida legítimo, pero deja que el módulo de matrículas conozca la persistencia de módulos ajenos, lo que se convierte en deuda en cuanto el sistema crece. La opción C resuelve problemas que este sistema no tiene —escala masiva, equipos múltiples, autonomía de despliegue— y rompe algo que sí tiene y que es crítico: la consistencia inmediata de la regla de cupo. Siguiendo el criterio de Newman (2021), mi-

grar a microservicios debe responder a un problema concreto identificado, y en un sistema académico de alcance institucional ese problema no está presente.

Conviene añadir que la opción B deja abierta la puerta a la C: un módulo que ya se comunica por puertos puede extraerse a un servicio independiente reemplazando el adaptador local por uno remoto, sin reescribir la lógica. Elegir B hoy no cierra el camino a C mañana, lo prepara.

16. Conclusiones

No existe una arquitectura superior, sino decisiones adecuadas a un contexto. Calificar doce estilos frente a seis atributos deja claro que ninguna fila gana en todas las columnas: Microservicios obtiene las mejores notas en escalabilidad y despliegue y la peor en complejidad operativa, mientras que la arquitectura en capas obtiene la mejor en costo de entrada y las peores en desacoplamiento. La pregunta útil no es cuál es mejor, sino qué atributo está realmente en juego en el sistema que se va a construir.

El criterio para trazar límites importa más que el estilo elegido. La diferencia entre un sistema modular y uno acoplado no depende de si se usan microservicios, sino de con qué criterio se cortan las partes. Partir por capas técnicas produce módulos que siempre cambian juntos; partir por contexto de negocio produce módulos que cambian por separado. Esta es la razón por la que DDD aparece como paso previo a cualquier partición seria, y por la que un monolito modular bien delimitado suele ser preferible a microservicios mal delimitados.

La complejidad distribuida se paga completa y por adelantado. El caso aplicado lo muestra con precisión: convertir la validación de cupo en una operación entre servicios no la hace más escalable, la hace más difícil, pasando de una verificación transaccional de tres líneas a requerir reserva de cupo o una saga con compensación. Esa complejidad solo se justifica cuando existe un problema de escala u organización que el monolito demostradamente no resuelve.

La decisión correcta es la que mantiene abiertas las opciones futuras al menor costo presente. Es la conclusión más transferible del informe: la arquitectura hexagonal con módulos se eligió no porque resuelva hoy un problema de escala, sino porque desacopla a costo bajo y prepara el camino hacia una eventual extracción de servicios. Una arquitectura que se puede evolucionar vale más que una que acierta el destino final desde el principio, porque en software el destino final casi nunca se conoce al empezar.

17. Referencias

Bass, L., Clements, P., & Kazman, R. (2021). *Software architecture in practice* (4th ed.). Addison-Wesley.

Cockburn, A. (2005). *Hexagonal architecture*. https://alistair.cockburn.us/hexagonal-architecture/

Erl, T. (2005). *Service-oriented architecture: Concepts, technology, and design*. Prentice Hall.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Hohpe, G., & Woolf, B. (2003). *Enterprise integration patterns: Designing, building, and deploying messaging solutions*. Addison-Wesley.

International Organization for Standardization. (2011). *ISO/IEC 25010:2011 — Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. ISO.

Krasner, G. E., & Pope, S. T. (1988). A cookbook for using the model-view controller user interface paradigm in Smalltalk-80. *Journal of Object-Oriented Programming*, 1(3), 26–49.

Martin, R. C. (2017). *Clean architecture: A craftsman's guide to software structure and design*. Prentice Hall.

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2nd ed.). O'Reilly Media.

Richards, M., & Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.

Roberts, M., & Chapin, J. (2017). *What is serverless?* O'Reilly Media.

Szyperski, C. (2002). *Component software: Beyond object-oriented programming* (2nd ed.). Addison-Wesley.

Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley.